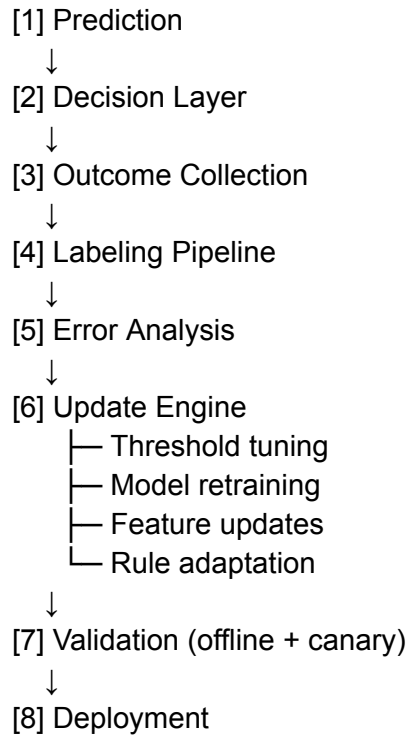




AGENTIC

Model predicts → system logs outcome → retrain later

ML services

Keep these as separate callable tools:

- `risk_model.predict(features)` → fraud probability
- `anomaly_model.score(features)` → novelty score
- `graph_model.lookup(entity_ids)` → cluster/ring risk
- `calibrator.predict(raw_score)` → calibrated probability

Agent tools

Expose the ML services and operational systems as tools:

- `get_transaction_context`
- `get_customer_profile`

- `score_risk_model`
- `score_anomaly_model`
- `get_graph_risk`
- `fetch_recent_cases`
- `run_policy_simulation`
- `create_candidate_rule`
- `deploy_canary`
- `rollback_policy`

Orchestrator

Represent the workflow as a graph:

$S_t = \{x_t, \text{scores}, \text{reasoning}, \text{proposal}, \text{approval}, \text{outcome}\}$
 $S_{t+1} = \{x_{t+1}, \text{scores}_{t+1}, \text{reasoning}_{t+1}, \text{proposal}_{t+1}, \text{approval}_{t+1}, \text{outcome}_{t+1}\}$

Each node updates shared state, and edges decide what runs next. That maps directly to LangGraph's model of shared state plus node/edge routing.

`ingest_event`

- > `build_features`
- > `run_ml_models`
- > `supervisor_agent`

`supervisor_agent`

- > `low_risk_path`
- > `high_risk_path`
- > `uncertain_path`
- > `drift_detection_path`

`high_risk_path`

- > `investigation_agent`
- > `policy_agent`
- > `decision`

`uncertain_path`

- > `investigation_agent`
- > `request_more_tools`
- > `policy_agent`
- > `decision`

drift_detection_path

-> collect_recent_outcomes

-> evaluation_agent

-> simulate_change

-> canary_or_reject